

07-June-2026

Agenda:

ML-5

- gradient descent → ●
- types of gradient descent ● optimization algo
- OLS (ordinary least squares) → ●
- practical (library)

Linear Regression using Gradient Descent:

Task/goal: to understand pattern using linear equations $\rightarrow$

data $\rightarrow$ $X \rightarrow$ features $\rightarrow$ count $\rightarrow$ linear equation
 $Y \rightarrow$ target

$$\textcircled{1} \quad x_1 \quad x_2 \quad x_3 \quad]$$

② $\begin{bmatrix} x_0 & x_1 & x_2 & x_3 \end{bmatrix}$
 $\downarrow$
 X

$\begin{bmatrix} m_0 \\ m_1 \\ m_2 \\ m_3 \end{bmatrix} \leftarrow \beta$ where $m_0 = C$
 $x_0 = 1$

② initialize $\beta \rightarrow \begin{matrix} \text{all} \rightarrow 0 \\ \text{randomly} \end{matrix}$ (step 1 - Algo)

④ $y_{\text{pred}} = X \cdot \beta$ (step-2)

⑤ error = $(y - y_{pred})$ (step-3)

⑥ calculate gradient $\Rightarrow$ direction rate of change (step-4) gradient formula $\leftarrow$ loss function

⑦ update weights (step-5)

$$\beta_{\text{new}} = \beta_{\text{old}} - \text{gradient} \times \text{learning rate}$$

$$\underline{\text{gradient}} = \frac{X^T \cdot (y_{\text{pred}} - y)}{n}$$

[step: 2-5 is in loop]

Type of gradient descent

(1) Batch gradient descent (default/vanilla)

$X \rightarrow$ all my training data

$$\text{gradient} = X^T @ (y_{\text{pred}} - y) / n$$

method $\rightarrow$ looks at entire dataset $\rightarrow$ calculate gradient
single update to weights $\leftarrow$

(2) Stochastic Gradient Descent (SGD)

$X[i] \rightarrow$ random data point from training data

$\rightarrow y_{\text{pred}} = X[i] \cdot \beta$

$\rightarrow \text{error} = y[i] - y_{\text{pred}}$

$\rightarrow \leftarrow \text{gradient}(y_{\text{pred}} - y[i])$

$\rightarrow$ update weights

update weights after looking at exactly one random row of data

(3) Mini-Batch Gradient Descent (Deep learning)

$X \rightarrow$ all data $\rightarrow$ Batch-GD

$X[0:16]$

$\downarrow$
16, 32, 64, 128 data point in a batch

$\rightarrow$ for i in range(0, 11): $\rightarrow$ 160 rows

for i in data: Batch $\rightarrow$ 16

len(i) $\rightarrow$ 16

$\rightarrow X \cdot \beta$ [16 data point]

$\rightarrow$ gradient

$\rightarrow$ update weight

$\rightarrow$ 10 iteration

$X \rightarrow$ all data $\rightarrow$ 100,000

$X[i] \rightarrow$ single row $\rightarrow$ 1

$\rightarrow$ epoch

11 $\rightarrow$ 11 $\times$ 10 $\rightarrow$ 110

10 $\rightarrow$ iteration

deep learning $\rightarrow$ images $\rightarrow$ [540 $\times$ 540 $\times$ 3] $\leftarrow$ single image

$\downarrow$
83 gb

$\downarrow$
874800 $\rightarrow$ int 8

$\rightarrow$ 0.33 mb / 854.3 kb

1 epoch $\rightarrow$ 10 iterations

$\vdots$

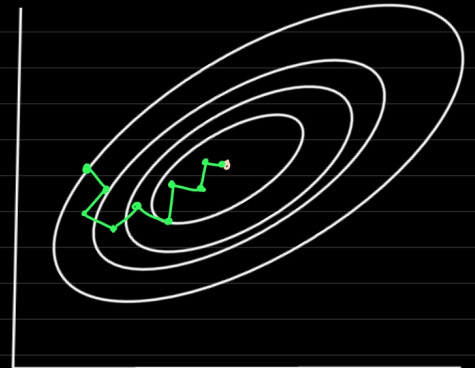
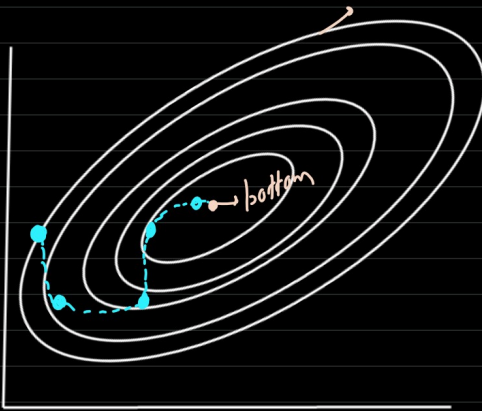
11 epoch $\rightarrow$ 10 iteration

$X \cdot \beta$ → data

batch = 2
= 4

batch = 1 → row
↓
→ stochastic GD

top of hill



Batch-GD

$x \rightarrow$ all
smooth-stable learning

SGD

for i in X :
weight updation
→ uses less memory
→ global solution
→ very fast

Mini-Batch SGD

perfect middle ground
→ combines both
SGD & BGD

100

→ 1 time
→ 1 time
→ 1 time

epoch → 100,000

→ 100,000 → SGD → iterations → 100,000

MSE → 9.78

100,000 × 100

epoch → 1 → comp b → 700

→ 2 → 600

→ 5 → 100

→ 10 → 50

→ 20 → 10

→ 25 → 5

1 hr ← → 26 → 7

→ 27 → 5

→ 28 → 6

← 72 epochs
left

100

early stopping →

CV →

Linear Regression

Gradient Descent

- parameter update in iteration
- medium-large dataset
- disadvantage [computation]
- $J(\beta) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
Modified MSE

OLS (Ordinary Least Squares)

- finds the optimal param in a single pass
- good for small-medium dataset
- disadvantage [large-dataset]
calculus
- expensive for huge no of parameter/feature
- $J(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2$
- sum of squared errors

OLS derivation [loss function]

$$\begin{aligned} J(\beta) &= (y - \hat{y})^2 \rightarrow (y - X\beta)^2 \\ &= (y - X\beta) \cdot (y - X\beta) \\ &= y^T y - 2\beta^T X^T y + \beta^T X^T X \beta \end{aligned}$$

$$\frac{\partial J}{\partial \beta} = \text{Derivate w.r.t } \beta$$

$$\frac{\partial}{\partial \beta} (y^T y) = 0$$

$$\frac{\partial}{\partial \beta} (-2\beta^T X^T y) = \underline{-2X^T y}$$

$$\frac{\partial}{\partial \beta} (\beta^T X^T X \beta) = \underline{2 X^T X \beta}$$

$$\frac{\partial J}{\partial \beta} = 0 \quad -2 X^T y + 2 X^T X \beta$$

set the derivation = 0

$$-2 X^T y + 2 X^T X \beta = 0$$

solving for β :

$$\cancel{2 X^T X \beta} = \cancel{2 X^T y} \quad \beta = \frac{X^T y}{X^T X}$$

$$\beta \rightarrow X^T y (X^T X)^{-1}$$

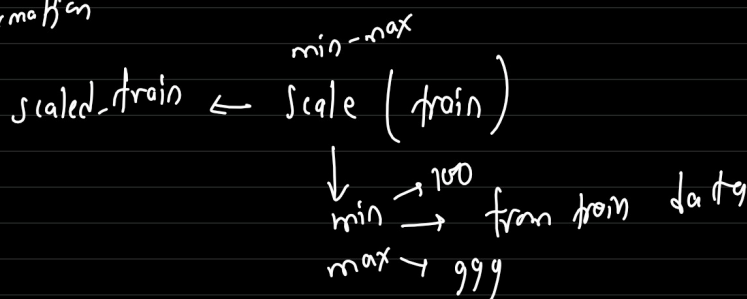
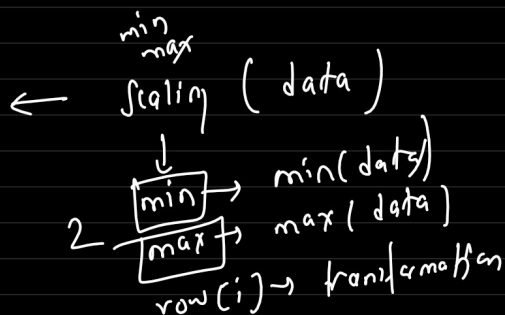
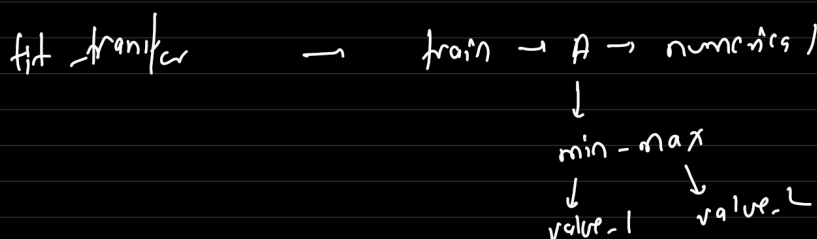
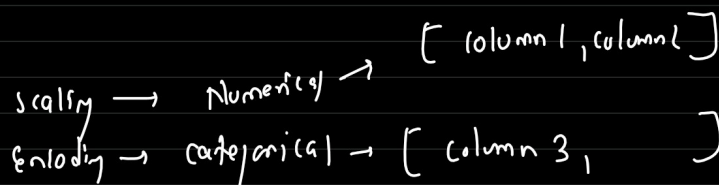
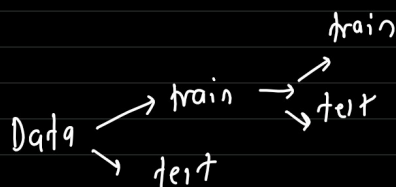
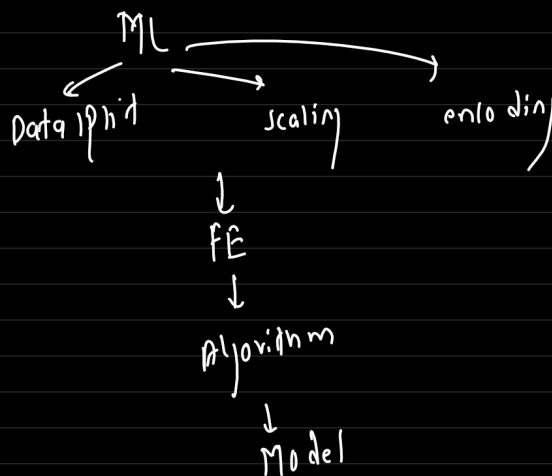
→ This is OLS closed-form solution

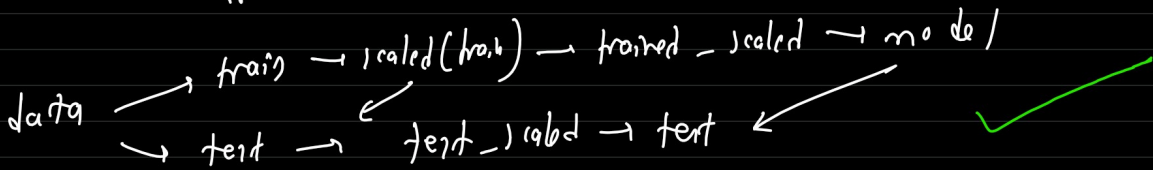
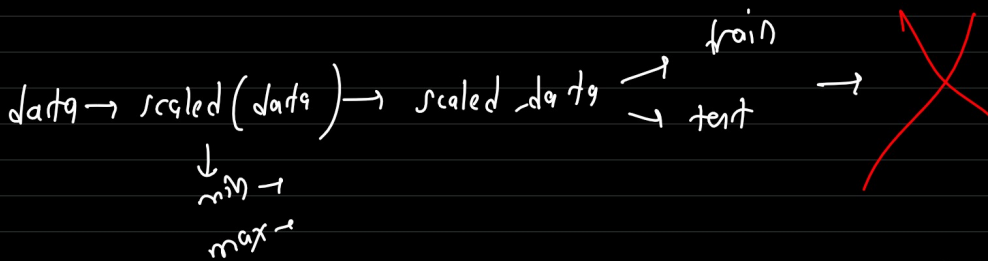
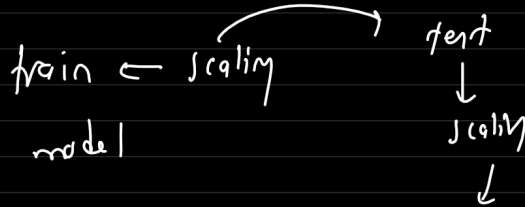
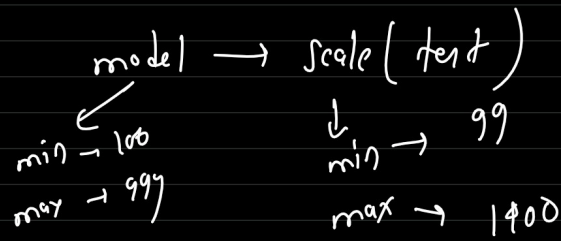
Gradient Descent:

$$\beta_{\text{new}} = \beta_{\text{old}} - \underset{\substack{\downarrow \\ \text{learning} \\ \text{rate}}}{\alpha} \cdot \frac{X^T (X \beta - y)}{n} \rightarrow \text{multiple times iteration}$$

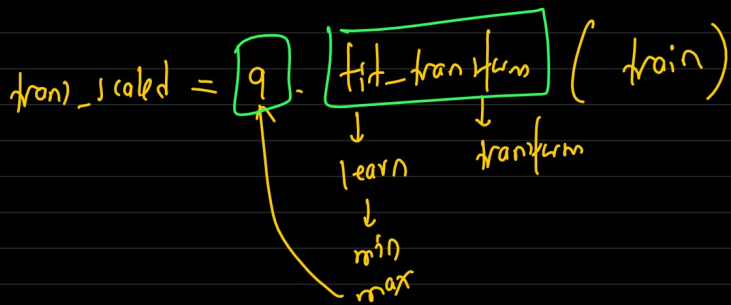
OLS:

$$\beta_{\text{new}} = (X^T X)^{-1} X^T y$$





$q = \text{Scaler}()$



$S = \text{Scaling}()$

$O = \text{OHE}()$

$\text{train_num_scale} = S.\text{fit_transform}(\text{train}[\text{numerical}])$

$\text{test_num_scale} = S.\text{transform}(\text{test}[\text{numerical}])$

$\text{train_cat_scale} = O.\text{fit_transform}(\text{train}[\text{cat}])$

$\text{test_cat_scale} = O.\text{transform}(\text{test}[\text{cat}])$

$x_1 \ x_2 \ x_3 \ x_4 \ \dots \ x_{14}$

$x_1 \rightarrow \text{min max scaling} \rightarrow 2 \rightarrow$
 x_2

$x_3 \rightarrow \text{stand scgn} \rightarrow 2$
 x_4

$x_5 \rightarrow \text{OHE} \rightarrow 2$
 x_6

$x_7 \rightarrow \text{Target enca} \rightarrow 2$
 x_8

column from $\left[\begin{array}{ccc} '1' & \boxed{} & \boxed{} \\ '2' & \boxed{} & \boxed{} \end{array} \right]$

fit $\rightarrow$
tr $\rightarrow$

